

Project Report: Clozr File Compressor

1 Introduction

The **Clozr File Compressor** is a comprehensive application for **file compression and decompression using Huffman Coding**.

It is designed to be:

- **User-friendly**, via a graphical user interface (GUI)
- **Flexible**, via a command-line interface (CLI)

The software supports:

- Text file compression (.txt)
- Binary file compression (.bin)
- Compression of multiple files into a single archive
- Visualization of the Huffman tree after compression
- Interactive display of the compression ratio (Ring or Bar)

2 Project Objectives

- Reduce file size to save storage space.
- Facilitate easier sharing of large files.
- Provide an **attractive and intuitive GUI** for casual users.
- Enable **CLI mode** for advanced users or server usage.
- Ensure robustness by **handling errors**, such as empty or missing files.

3 System Requirements

3.1 Software Requirements

- Python 3.10 or higher
- Operating system: Windows / Linux / MacOS

3.2 Core Python Libraries

- PyQt5 : for building the GUI
- json : for storing and loading Huffman codes and metadata
- os : for file and directory management

3.3 Optional Libraries

- qdarkstyle : for dark mode in the GUI

4 Technical Description

4.1 Project Structure

The project is organized into modules:

Module	Purpose
compressor	Compress text, binary, or multiple files
decompressor	Decompress files and restore original data
GUI	Interactive graphical user interface
utils	Helper tools like Logger, compression stats, visualizer, exceptions

4.2 Compression Module

4.2.1 Text File Compression (*compress_text_file*)

- Reads the text file completely
- Converts characters to a string format suitable for Huffman compression
- Builds a **frequency map** for each character
- Constructs the **Huffman tree**
- Generates **Huffman codes**
- Encodes the data (encode_data) into a binary stream
- Saves the compressed data as .bin
- Saves Huffman codes in .json
- Visualizes the Huffman tree as .png

4.2.2 Binary File Compression (*compress_binary_file*)

- Same steps as text compression but directly handles bytes
- Supports very large files

4.2.3 Multiple File Compression (*compress_multiple_files*)

- Merges contents of all files into a temporary string
- Builds a single Huffman tree for all files
- Stores the length of each file in **metadata**
- Compresses the data into a single .bin archive
- Saves Huffman codes and metadata as `_meta.json`
- Allows extraction of individual files during decompression

4.3 Decompression Module

4.3.1 Rebuilding the Huffman Tree (*rebuild_tree_from_codes*)

- Reconstructs the original tree from the stored codes
- Enables accurate decoding of compressed data

4.3.2 File Decompression

- **Text/Binary Files:** `decompress_text_file` and `decompress_binary_file`
 - Reads the .bin file
 - Removes padding bits
 - Uses the Huffman tree to decode data
 - Saves the original file
- **Multiple Files Archive:** `decompress_archive`
 - Uses `_meta.json` to know the length of each original file
 - Splits the decompressed data according to the metadata
 - Saves each file in the specified output folder

4.4 Graphical User Interface (GUI)

4.4.1 Features

- Select **operation mode**: Compress or Decompress
- Select **file type**: Text, Binary, or Multiple Files

- Choose input files and output path
- Display **progress bar** for operation progress
- Show **compression ratio** dynamically (Ring or Bar)
- Display **Huffman tree image** after compression

4.4.2 Design

- Dark background color (#1C2B39)
- Bright green buttons (#00F5A0)
- Clear and readable text
- TextEdit widget for **log messages**
- QSplitter to display Huffman tree beside logs

4.5 Command-Line Interface (CLI)

- Run via:

```
python main.py --cli
```

- CLI Options:
 - Compress a text file
 - Decompress a file
- Displays the **compression ratio** after operation
- Suitable for running on servers or environments without GUI

5 Algorithm

The software implements **Huffman Coding**:

1. Calculate the **frequency of each character/byte** in the data
2. Build a **binary Huffman tree**, placing less frequent symbols at deeper nodes
3. Generate a **unique binary code** for each character/byte
4. **Encode the data** as a binary string
5. Store compressed data along with Huffman codes and metadata (for multi-file archives)

6 Project Outputs

- **Compressed files:** .bin
- **Huffman codes:** .json
- **Huffman tree images:** .png
- **Multi-file archive:** _meta.json + .bin
- **Compression ratio:** displayed in GUI and CLI

7 Analysis and Results

- Best compression ratio achieved on text files with high repetition
- Binary files may achieve variable compression ratios depending on content
- Robust error handling for:
 - Empty files
 - Missing files
 - Errors during compression/decompression

8 Conclusion

- The project provides a **complete solution for Huffman-based file compression and decompression**
- Supports **GUI and CLI**, making it **flexible and professional**
- Future improvements could include **image, audio, and video compression support**